

Техническое задание: агрегатор объявлений о недвижимости «Real Estate Pro»

«Проектирование и разработка архитектуры сервисов и приложений». Команда: Матвеев Д., Голубев И., Будаев Д., Гаджигасанов Т.

1. Общие сведения

1.1. Назначение. Сервис собирает объявления об аренде и продаже квартир с нескольких площадок и из разных стран, приводит их к единому формату и отдаёт через единый поиск с фильтрами. Пользователь может сохранить поиск (например, «1-к в Минске до \$400») и мгновенно получить уведомление в Telegram, когда появится подходящее объявление.

1.2. Цель проекта. Разработать backend на микросервисной архитектуре (3 сервиса) с PostgreSQL, Redis (кэш, Pub/Sub, дедупликация) и развёртыванием через docker-compose, а также простой веб-интерфейс, демонстрирующий работу системы.

1.3. Целевая аудитория. Люди, ищущие жильё в аренду или на покупку, в том числе в разных странах.

1.4. Правовые и этические требования.

- Сервис не копирует объявления целиком: хранятся нормализованные поля и ссылка на первоисточник, который показывается в каждой карточке.
- Сбор данных ведётся с умеренной частотой, с учётом robots.txt и условий использования площадок.
- Из персональных данных хранятся только Telegram chat_id и параметры подписок; пользователь может удалить подписки и учётную запись командой бота.
- Для демонстрации используется mock-режим на локальных JSON-дампах (см. 3.4).

2. Функциональные требования

В системе три роли: **посетитель** (поиск без регистрации через веб-интерфейс), **подписчик** (пользователь Telegram-бота с сохранёнными поисками) и **администратор** (служебные debug-эндпоинты).

2.1. Сбор данных (collector)

- ФТ-1.1. Источники подключаются через единый интерфейс адаптера: `fetch()` -> `list[RawListing]`. Минимум три адаптера: два реальных источника и один mock.
- ФТ-1.2. Режим mock включается переменной `MOCK_MODE=true`: адаптер читает локальные JSON-дампы (500+ объектов из разных стран). При ошибке реального источника (блокировка, капча, таймаут) сервис автоматически переключается на mock для этого источника.
- ФТ-1.3. Нормализация: конвертация цены в базовую валюту (USD) с сохранением исходной цены и валюты, очистка текста от HTML, приведение комнатности к числу (студия = 0), площади к м², страны и города к справочникам.
- ФТ-1.4. Сбор запускается по расписанию (период настраивается) и вручную через `POST /collect/run`.
- ФТ-1.5. Перед отправкой в Core API выполняется быстрая дедупликация по хэшу в Redis (см. 3.3).
- ФТ-1.6. Нормализованные объявления отправляются в Core API запросом `POST /api/v1/listings`. При недоступности Core API отправка повторяется с экспоненциальной задержкой.
- ФТ-1.7. Веб-интерфейс (Jinja2 + HTMX) из двух страниц: главная с фильтрами и списком карточек, страница карточки объявления со ссылкой на первоисточник. Интерфейс работает через Core API.

2.2. Основной API (core-api)

- ФТ-2.1. `POST /api/v1/listings` сохраняет объявление. Повторная отправка того же объявления не создаёт дубль: возвращается существующая запись (идемпотентность).
- ФТ-2.2. `GET /api/v1/listings` возвращает список с фильтрами: тип сделки (аренда/покупка), страна, город, цена от/до (в USD), число комнат, площадь от/до, источник; сортировка по цене и дате; пагинация (`page`, `size`, не более 100 на странице).
- ФТ-2.3. `GET /api/v1/listings/{id}` возвращает карточку объявления.
- ФТ-2.4. Справочники: `GET /api/v1/cities`, `/countries`, `/sources`, `/currencies`.
- ФТ-2.5. После успешного сохранения нового объявления сервис публикует событие в канал Redis Pub/Sub `listings.new` (формат в 4.3).
- ФТ-2.6. Ответы на частые запросы кэшируются в Redis; кэш сбрасывается при появлении новых объявлений (см. 3.5).

- ФТ-2.7. `GET /health` проверяет доступность Postgres и Redis. Swagger-документация доступна на `/docs`.

2.3. Уведомления и пользователи (notifier)

- ФТ-3.1. Telegram-бот: `/start` регистрирует пользователя, `/subscribe` создаёт подписку (диалог или параметры: тип сделки, страна, город, комнаты, максимальная цена), `/list` показывает подписки, `/unsubscribe <id>` удаляет подписку, `/stop` удаляет пользователя и все его данные.
- ФТ-3.2. Пользователи и подписки хранятся в PostgreSQL (схема `notifier`).
- ФТ-3.3. Сервис подписан на канал `listings.new` и обрабатывает события в горутинах (пул воркеров).
- ФТ-3.4. Для каждого события проверяются все подписки; при совпадении всех заданных критериев пользователю отправляется сообщение с ценой, адресом, комнатностью и ссылкой на первоисточник.
- ФТ-3.5. Одному пользователю одно объявление отправляется не более одного раза (журнал отправленных уведомлений).
- ФТ-3.6. Если `TELEGRAM_BOT_TOKEN` не задан, сервис работает в режиме «лог»: совпадения пишутся в `stdout` вместо отправки. Это позволяет запускать систему без бота.
- ФТ-3.7. `GET /health` для проверки состояния сервиса; служебный `POST /debug/event` для имитации события (только для демонстрации).

3. Бизнес-правила

3.1. Единый формат объявления

Обязательные поля: источник, внешний ID, ссылка на первоисточник, тип сделки (`rent` / `sale`), страна, город, цена в исходной валюте, валюта, цена в USD, число комнат. Необязательные: адрес, площадь, этаж, описание, фото (ссылки), дата публикации на источнике. Объявления без обязательных полей отклоняются с кодом 422.

3.2. Валюты

Базовая валюта — USD. Курсы хранятся в таблице `currencies` и задаются в конфигурации (статичные, для воспроизводимости демонстрации); цена в USD округляется до целого.

3.3. Дедупликация

Два уровня защиты:

- **Хэш ссылки:** SHA-256 от пары (источник, внешний ID или нормализованный URL). Хранится в Redis (SET) и в Postgres с ограничением UNIQUE. Совпадение означает, что это то же объявление.
- **Хэш содержимого:** SHA-256 от нормализованных (город, адрес, комнаты, площадь, тип сделки) для поиска одной и той же квартиры на разных площадках. Такие объявления связываются, а не отбрасываются.

3.4. Моск-режим

Моск-дамп содержит не менее 500 объявлений для 5+ стран и 10+ городов, с реалистичным разбросом цен и комнатности. Формат дампа совпадает с форматом ответа реального адаптера, поэтому остальной конвейер не знает, какой источник работает.

3.5. Кэш

Ключ кэша — хэш нормализованных параметров запроса. Время жизни 10 минут. При сохранении нового объявления сбрасываются ключи списка (по версии кэша: счётчик `cache:version` увеличивается, старые ключи истекают сами).

3.6. Параметры

Итоговые значения уточняются после тестирования.

Параметр	Значение
TTL кэша списков	10 минут
Размер страницы по умолчанию / максимум	20 / 100
Период запуска сбора	15 минут
Повторы отправки в Core API	5, задержка 1–30 с
Подписок на пользователя	до 10
Уведомлений пользователю	не более 30 в час
Воркеров обработки событий	8
Часовой пояс сервиса	Europe/Minsk

4. Архитектура

Система состоит из 3 микросервисов, одного PostgreSQL (отдельная схема на сервис) и одного Redis.

4.1. Микросервисы

Сервис	Стек	Ответственность	Порт	Ответственный
core-api	Python, FastAPI, SQLAlchemy, Alembic	Хранилище объявлений, поиск, кэш, публикация событий	8000	Тимур БД: Илья
collector	Python, httpx, BeautifulSoup, Jinja2, HTMX	Адаптеры источников, нормализация, mock, веб-интерфейс	8001	Тимур UI, данные: Дима
notifier	Go, go-redis, Telegram Bot API	Пользователи, подписки, подбор и уведомления	8080	Даниил

4.2. Схема данных

Схема core (владелец — core-api): `countries`, `cities` (FK на страну), `sources`, `currencies` (код, курс к USD), `listings` (FK на город, источник, валюту; поля из 3.1; `url_hash` UNIQUE, `content_hash`, `created_at`). Индексы: (`deal_type`, `city_id`, `price_usd`), (`rooms`), (`created_at` DESC), уникальный `url_hash`.

Схема notifier (владелец — notifier): `users` (telegram chat_id, дата создания), `subscriptions` (FK на пользователя, критерии, активность), `notifications_log` (пара пользователь + объявление, уникальная).

Каждый сервис работает только со своей схемой; данные других сервисов получают через API или события.

4.3. Redis

- **Кэш** списков объявлений (`cache:listings:*`, TTL 10 минут).
- **Дедупликация** (`dedup:urls`, множество хэшей).
- **Pub/Sub**: канал `listings.new`.

Сообщение в `listings.new` — JSON-объект, в котором есть всё необходимое для подбора без обращения к Core API: `id`, `deal_type`, `country`, `city`, `rooms`, `price`, `currency`, `price_usd`, `area_m2`, `address`, `url` (первоисточник), `source`,

`created_at`. Названия полей и их типы фиксируются в README и не меняются без согласования команды.

4.4. Поток данных

collector → `POST /api/v1/listings` → core-api (Postgres, сброс кэша) → Redis
`listings.new` → notifier → Telegram. Веб-интерфейс collector читает данные через `GET /api/v1/listings`.

5. Нефункциональные требования

5.1. Безопасность

- НФТ-1. Секреты (токен Telegram, пароли БД) задаются через `.env` и не хранятся в Git; в репозитории лежит `.env.example`.
- НФТ-2. SQL-запросы выполняются через ORM или с параметрами. Входные данные валидируются (Pydantic).
- НФТ-3. Служебные эндпоинты (`/debug/*`, `/collect/run`) защищены ключом из переменной окружения.
- НФТ-4. Порты Postgres и Redis публикуются наружу только для разработки.

5.2. Надёжность и хранение данных

- НФТ-5. Данные Postgres сохраняются при перезапуске контейнеров (volume).
- НФТ-6. Структура БД изменяется только через миграции (Alembic для схемы `core`, SQL-миграции для `notifier`).
- НФТ-7. Если Redis недоступен, core-api продолжает отвечать из Postgres без кэша; если недоступен Core API, collector повторяет отправку.
- НФТ-8. Для всех контейнеров настроены healthcheck и порядок запуска через `depends_on`.
- НФТ-9. Внешние HTTP-вызовы выполняются с таймаутом; клиент получает понятную ошибку, а не зависший запрос.

5.3. Производительность

- НФТ-10. Список объявлений на базе из 500+ записей отвечает быстрее 300 мс (кэш) и быстрее 500 мс (без кэша).
- НФТ-11. Уведомление отправляется пользователю не позднее 5 секунд после появления события.

5.4. Разработка и сопровождение

НФТ-12. Ошибки API возвращаются в едином JSON-формате {error, message} с корректным HTTP-кодом.

НФТ-13. Вся система запускается одной командой `docker compose up --build`; конфигурация задаётся через `.env`.

НФТ-14. Код хранится в Git-репозитории; README описывает запуск и структуру проекта.

5.5. Интерфейс

НФТ-15. Веб-интерфейс корректно работает в актуальных версиях браузеров.

6. Распределение работ

6.1. Илья Голубев: PostgreSQL и сопровождение БД

Проектирует и поддерживает обе схемы: `core` и `notifier`; таблицы, связи, ограничения и индексы. Готовит Alembic-миграции для `core` и SQL-миграции для `notifier`, начальные справочники, ER-диаграмму и описание полей. Настраивает роли и права доступа, проверяет планы запросов, готовит инструкции резервного копирования и восстановления. Изменения структуры выполняются через миграции, согласованные с владельцами сервисов.

6.2. Дима Будаев: интерфейс, данные и ручная проверка

По готовым примерам оформляет HTML/CSS и Jinja2-шаблоны двух страниц: список с фильтрами и карточка объявления. Подготавливает примеры и проверяет mock-дамп: минимум 500 объектов, 5 стран, 10 городов. Ведёт чек-лист ручных проверок, описывает найденные ошибки, актуализирует инструкцию запуска и сценарий демонстрации.

6.3. Тимур Гаджигасанов: Python-бэкенд

Разрабатывает `core-api`: валидацию, запись, поиск, пагинацию, справочники, Redis-кэш и публикацию событий. Реализует логику `collector`: адаптеры, нормализацию, дедупликацию, расписание, повторы, mock-режим и подключение веб-шаблонов. Согласует модели и запросы с Ильёй.

6.4. Даниил Матвеев: notifier и интеграция

Разрабатывает Go-сервис, команды Telegram-бота, подписки, подбор объявлений и журнал уведомлений. Отвечает за Docker Compose, конфигурацию, healthcheck и сквозную интеграцию. SQL-миграции `notifier` готовит Илья; код доступа к данным и выполнение миграций при развёртывании согласуются с Даниилом.

6.5. Границы ответственности

База данных. Илья отвечает за структуру и сопровождение PostgreSQL в целом. На уровне приложения core-api обращается только к core, notifier только к notifier. Collector получает и передаёт данные через API. notifications_log хранит ID объявления из события без межсхемного внешнего ключа на core.listings.

Изменения БД. Разработчик сервиса описывает требуемое изменение; Илья готовит миграцию и согласует её с ним. Разработчик обновляет модели и запросы. Даниил проверяет порядок применения миграций при общем запуске. Ручные изменения структуры вне миграций не используются.

Redis. Тимур отвечает за кэш, дедупликацию и публикацию listings.new; Даниил - за получение события и обработку уведомлений. Настройка Redis в Compose относится к Даниилу. Это отдельная от сопровождения PostgreSQL зона работ.

Интерфейс и данные. Тимур предоставляет Диме рабочие маршруты, пример шаблона и JSON-формат. Дима оформляет страницы и готовит примеры; разработчик реализует загрузчик/генератор дампа и автоматическую проверку формата. Илья проверяет согласованность справочников.

Контракты. Форматы API, RawListing и listings.new фиксируются до параллельной разработки. Изменения согласуют затронутые участники. Дима обновляет README по проверенным сведениям; техническую точность проверяют владельцы сервисов.